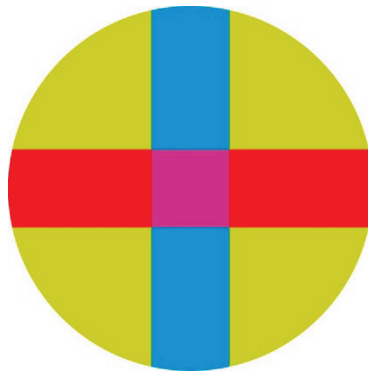


UNIVERSIDAD SAN PABLO - CEU

ESCUELA POLITÉCNICA SUPERIOR

GRADO EN INGENIERÍA DE SISTEMAS DE INFORMACIÓN



TRABAJO FIN DE GRADO

Arbitraje de Esgrima por IA

Fencing Arbitration by AI

Autor: Alejandro Ramírez Larena

Tutor: Constantino Antonio García Martínez

Julio 2024



UNIVERSIDAD SAN PABLO-CEU

ESCUELA POLITÉCNICA SUPERIOR

División de Ingeniería

Calificación del Trabajo Fin de Grado

Datos del alumno

NOMBRE:

Datos del Trabajo

TÍTULO DEL PROYECTO:

Tribunal calificador

PRESIDENTE:

FDO.:

SECRETARIO:

FDO.:

VOCAL:

FDO.:

Reunido este tribunal el ____/____/_____, acuerda otorgar al Trabajo Fin de Grado presentado por D./Dña. _____ la calificación de _____

Resumen

En la actualidad, la Inteligencia Artificial (IA) desempeña un papel cada vez más importante en diversos ámbitos de nuestra vida, incluyendo el del deporte. Este trabajo se centra en el desarrollo de un modelo de video arbitraje asistido por IA aplicado a la esgrima, que tiene como objetivo mejorar la precisión y la consistencia en la toma de decisiones arbitrales.

Durante la ejecución del proyecto, se han utilizado herramientas de análisis de poses humanas, como OpenPose, para extraer las poses de los tiradores. Las poses extraídas han servido como entrada en un modelo de aprendizaje profundo, diseñado con una arquitectura de redes neuronales que incluye capas LSTM (Long Short-Term Memory) y capas densas. Asimismo, se han implementado diversas técnicas de aumento de datos y se ha llevado a cabo una búsqueda exhaustiva de hiperparámetros para optimizar el rendimiento del modelo. A pesar de estos esfuerzos, los resultados obtenidos no han sido satisfactorios. La precisión en la validación sólo alcanzó un 60,93%, lo cual indica que el modelo no logra un desempeño adecuado para el propósito de este proyecto.

Los resultados de este estudio subrayan la necesidad de un enfoque mejorado. Para futuras investigaciones, es crucial considerar significativamente la expansión del conjunto de datos utilizado, así como integrar la detección de posiciones humanas con el reconocimiento de la posición del arma de los tiradores. Además, se sugiere la posibilidad de involucrar a múltiples expertos en la validación y catalogación de los videos para mejorar la precisión y efectividad del modelo en próximas investigaciones.

Palabras Clave

Inteligencia Artificial, Vídeo Arbitraje, Esgrima, Florete, Aprendizaje Profundo, Precisión, Consistencia, Decisiones Arbitrales

Abstract

Currently, Artificial Intelligence (AI) plays an increasingly important role in various aspects of our lives, including sports. This work focuses on developing an AI-assisted video refereeing model applied to fencing, which aims to improve the accuracy and consistency of refereeing decisions.

During the project execution, human pose analysis tools, such as OpenPose, were used to extract the fencers' poses. The extracted poses served as input to a deep learning model, designed with a neural network architecture that includes LSTM (Long Short-Term Memory) layers and dense layers. Additionally, various data augmentation techniques were implemented, and an exhaustive hyperparameter search was carried out to optimize the model's performance. Despite these efforts, the results obtained were not satisfactory. The validation accuracy only reached 60.93%, indicating that the model does not achieve adequate performance for the purpose of this project.

The results of this study highlight the need for an improved approach. For future research, it is crucial to consider significantly expanding the dataset used, as well as integrating human position detection with the recognition of the fencers' weapon position. Furthermore, it is suggested to involve multiple experts in the validation and cataloging of videos to improve the accuracy and effectiveness of the model in future investigations.

Keywords

Artificial Intelligence, Vídeo Refereeing, Fencing, Foil, Deep Learning, Accuracy, Consistency, Refereeing Decisions.

Índice de contenidos

Capítulo 1 Introducción	1
1.1 Introducción y motivación	1
1.2 El arte del florete	2
Capítulo 2 Gestión del proyecto	5
2.1 Modelo de Ciclo de Vida	5
2.2 Fases del Proyecto	5
2.2.1 Fase de Arranque y Obtención de Datos.....	5
2.2.2 Fase de Configuración y Extracción de Poses.....	7
2.2.3 Fase de Desarrollo y Entrenamiento del Modelo.....	9
Capítulo 3 Materiales y métodos	11
3.1 Preprocesamiento de Datos	11
3.2 Redes Neuronales	13
3.2.1 Capas y arquitecturas habituales	14
3.2.2 Funciones de Activación.....	14
3.2.3 Técnicas de Optimización y Regularización.....	15
3.2.4 División de Conjuntos de Datos.....	17
3.2.5 Optimizadores	18
3.2.6 Métricas de Evaluación	19
Capítulo 4 Diseño e implementación	21
4.1 Recursos del proyecto	21
4.2 Preprocesamiento de datos.....	21
4.2.1 Arranque y creación del dataset	21
4.2.2 Obtención de las poses	22
4.2.3 Filtrado de poses no válidas	23
4.2.4 Formateo y ordenación de las poses.....	24
4.3 División de datasets usados para el entrenamiento.	25
4.4 Creación de Dataloader	25
4.5 Diseño de arquitectura	27
4.6 Entrenamiento con Optuna	28
4.7 Entrenamiento final.....	30
Capítulo 5 Resultados	31
5.1 Resultados de script de filtrado	31

5.2 Resultados de las transformaciones	33
5.3 Resultados de entrenamiento	35
Capítulo 6 Conclusiones	38
6.1 Resumen del Trabajo	38
6.2 Limitaciones y oportunidades futuras	38
6.2.1 Limitaciones Actuales	38
6.2.2 Oportunidades Futuras	39
Capítulo 7 Bibliografía.....	41
Glosario de términos	43

Índice de figuras

<i>Figura 1 Áreas de tocado válido [4]</i>	2
<i>Figura 2 Historias de usuario primera fase</i>	7
<i>Figura 3 Historias de usuario segunda fase</i>	9
<i>Figura 4 Historias de usuario tercera fase</i>	10
<i>Figura 5 Identificación de poses en un fotograma</i>	12
<i>Figura 6 Distribución de pares de poses detectados</i>	12
<i>Figura 7 Funciones de activación [S. Rahman]</i>	15
<i>Figura 8 Representación de puntos clave [7]</i>	22
<i>Figura 9 Formula Binary Cross-Entropy [6]</i>	27
<i>Figura 10 Modelo de capas</i>	28
<i>Figura 11 Extracción de poses de un vídeo con OpenPose</i>	31
<i>Figura 12 Detección errónea de una pose en videomarcador</i>	32
<i>Figura 13 Detección errónea de poses de otros esgrimistas del fondo</i>	32
<i>Figura 14 Detección errónea del arbitro</i>	33
<i>Figura 15 Transformación de zoom</i>	34
<i>Figura 16 Transformación desplazamiento eje Y</i>	34
<i>Figura 17 Gráfico Accuracy (BinaryCross-Entropy)</i>	37

Índice de tablas

<i>Tabla 1 Rangos hiperparámetros.....</i>	<i>29</i>
<i>Tabla 2 Validación transformaciones y tiempo de ejecución.....</i>	<i>35</i>
<i>Tabla 3 Resultados.....</i>	<i>35</i>

Capítulo 1 Introducción

1.1 Introducción y motivación

La era digital ha contribuido en muchos ámbitos a desarrollar y mejorar nuestra sociedad. Cada vez es mayor el uso de tecnología en áreas como el deporte. Dentro de este campo, el uso de la inteligencia artificial crece de forma exponencial en la creación de modelos de aprendizaje y de optimización de técnicas deportivas [8].

Dentro de los deportes, la esgrima en particular es un deporte con reglas complejas que exigen, en el contexto competitivo, la toma de decisiones ágiles y certeras por parte del árbitro. Estas decisiones en ocasiones expuestas a controversia y polémica.

El sistema de arbitraje actual en la esgrima está expuesto a errores humanos en momentos cruciales, así como a la disparidad de criterios de arbitraje a lo largo de la competición, poniendo en riesgo la equidad y justicia de los resultados finales.

Este proyecto propone un sistema de vídeo arbitraje asistido por Inteligencia Artificial (IA) que tiene como objetivo mejorar la precisión y la consistencia en la toma de decisiones arbitrales.

El modelo se basa en el análisis automático de vídeos mediante técnicas de aprendizaje profundo. La IA puede analizar vídeos capturados durante la competición e identificar si el tocado corresponde al tirador de la derecha o al de la izquierda, creando información que puede servir de ayuda al árbitro en la toma de decisiones o análisis de errores, ofreciéndole información sobre las jugadas controvertidas.

Los principales objetivos del sistema propuesto son alcanzar una mayor precisión en el arbitraje y una mayor consistencia en las decisiones. La IA puede detectar eventos que el ojo humano podría pasar por alto, así como aplicar los criterios de forma uniforme a todas las jugadas, eliminando la subjetividad y la disparidad de criterios.

En definitiva, este proyecto persigue la creación de una solución innovadora para mejorar el arbitraje en la esgrima, con el potencial de contribuir a un deporte más justo y equitativo.

1.2 El arte del florete

El florete, una de las tres armas que componen el arte de la esgrima junto a la espada y el sable, se distingue por su elegancia, precisión y virtuosismo técnico. Esta arma, diseñada para asestar tocados mediante estocadas, se considera el arma base de la esgrima debido a su ligereza y flexibilidad, convirtiéndola en un instrumento ideal para desarrollar las habilidades y fundamentos de este deporte.

En un asalto de florete, los tocados se registran gracias a un peto metálico que delimita la zona válida a tronco y barbada, a diferencia de la espada y el sable como se puede ver en la *Figura 1*. Cualquier toque fuera de esta área no tiene validez y detiene el combate sin otorgar puntos. Esta regla, sumada a la prohibición de cubrir un blanco válido con uno no válido, garantiza la integridad y transparencia de la competición.



Figura 1 Áreas de tocado válido [4]

Los asaltos de florete se rigen por convenciones específicas que eliminan la posibilidad del "tocado doble". En caso de que ambos tiradores se toquen simultáneamente, el punto se otorga al que tiene la "prioridad", la cual recae en el atacante hasta que la pierde a través de una parada o un batimiento. Esta dinámica de parada y respuesta, junto con la determinación de la prioridad, añade un nivel de complejidad adicional al arbitraje de los combates de florete. Cada acción y reacción debe ser evaluada cuidadosamente para garantizar una competencia justa y precisa [20].

Capítulo 2 Gestión del proyecto

2.1 Modelo de Ciclo de Vida

En la realización de este proyecto, se ha seguido un proceso de gestión que combina elementos de metodologías ágiles y tradicionales. El enfoque adoptado fue iterativo y evolutivo, característico de las metodologías ágiles, donde el trabajo se desarrolló en entregas incrementales revisadas de forma continuada.

Las tareas se dividieron en unidades pequeñas y manejables, lo que permitió un progreso más organizado y controlado. Esta división facilitó una adaptación flexible a los cambios y a los comentarios recibidos durante las revisiones por parte del tutor, y también facilitó el proceso de validación continua durante todo el proyecto.

2.2 Fases del Proyecto

El desarrollo del proyecto se dividió en tres fases principales: arranque y obtención de datos, configuración y extracción de poses, y desarrollo y entrenamiento del modelo.

2.2.1 Fase de Arranque y Obtención de Datos

En esta primera fase, se realizaron diversos pasos que se describirán a continuación, para obtener y preparar una base de datos de vídeos de esgrima, centrados específicamente en la modalidad de florete.

En primer lugar, se procedió a localizar y obtener una base de datos que contuviera vídeos de esgrima. En paralelo, se llevó a cabo una investigación sobre diferentes alternativas y métodos para la extracción de poses, evaluando sus

ventajas y desventajas en comparación con la herramienta OpenPose, una herramienta de código abierto para la detección de poses humanas en imágenes y vídeos. Confirmada la elección de OpenPose, se descargó, instaló y configuró adecuadamente, asegurándose de que estuviera lista para procesar los vídeos descargados. Estos conceptos y la argumentación de los mismos se pueden ver en el Capítulo 3, específicamente en el punto 3.1.

Una vez que OpenPose estuvo listo, se identificaron y separaron los vídeos específicos de la modalidad de florete dentro de la base de datos obtenida, descartando otras modalidades como sable y espada. Posteriormente, los vídeos seleccionados fueron descargados para su análisis y posterior procesamiento, para la extracción de poses, y generación de archivos JSON con la información de las poses detectadas a lo largo del vídeo.

Para asegurar la calidad del proceso, se llevó a cabo una validación de la extracción de poses, revisando manualmente algunos resultados. Se identificaron y eliminaron del dataset aquellos JSON que eran demasiado pequeños, ya que estos no aportaban información relevante debido a que se trataban de vídeos corruptos o mal formados.

Para validar que los JSON eran correctos, se creó un script que convertía las poses de nuevo a vídeo, permitiendo verificar la correcta extracción de las poses.

Estas fases descritas en esta primera parte, son producto de las siguientes historias de usuario, que se pueden ver en la siguiente *Figura 2*.

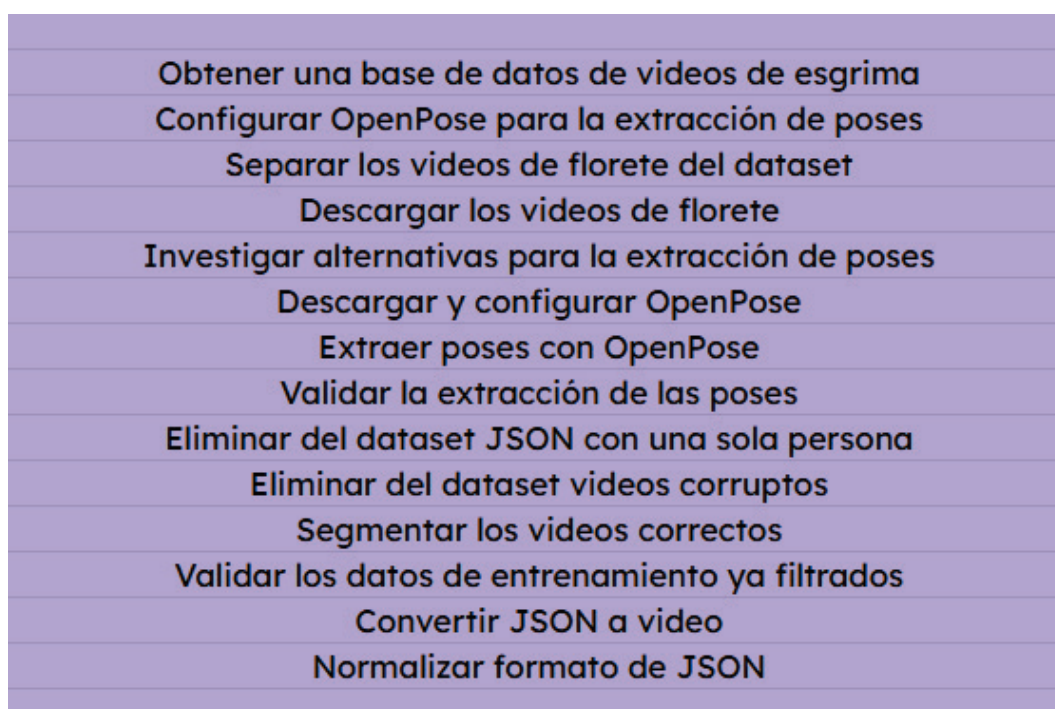


Figura 2 Historias de usuario primera fase

2.2.2 Fase de Configuración y Extracción de Poses

En la siguiente fase, se llevaron a cabo una serie de pasos para entrenar un modelo de aprendizaje profundo utilizando los datos de poses extraídos previamente. Para ello, se crearon tanto un entorno de trabajo en Google Colab como un entorno local utilizando Anaconda.

El primer paso fue cargar los datos de entrenamiento, los cuales consistían en los archivos JSON generados en la fase anterior. A continuación, se diseñó la forma del tensor, especificando la estructura y dimensiones adecuadas para los datos de entrada y salida del modelo.

Con la forma del tensor definida, se procedió a diseñar la arquitectura de capas del modelo. Esto implicó seleccionar y configurar las diferentes capas de la red neuronal, asegurando una estructura adecuada para la tarea de detección de poses.

Durante el proceso de diseño y optimización, se visualizaron los datos de entrenamiento para asegurar que las transformaciones y normalizaciones aplicadas fueran correctas. Se subió todo el código a un repositorio de Git y los keypoints (puntos clave) a OneDrive.

El siguiente paso fue cargar el dataset en el entorno de trabajo, tanto en Colab como en el entorno local. Se normalizaron los tensores para garantizar que los datos de entrada al modelo estuvieran en un rango adecuado, y se validaron estos tensores normalizados para asegurar la consistencia de los datos.

Para manejar los datos de manera eficiente durante el entrenamiento, se creó un Dataloader personalizado. Además, se implementaron diversas técnicas de aumento de datos para mejorar la robustez del modelo, tales como RandomZoom, HorizontalFlip y desplazamientos en los ejes X e Y. Cada una de estas transformaciones fue implementada y validada para asegurar su correcta aplicación. Todo esto se explicará más adelante en el Capítulo 4, además se pueden observar las historias de usuario de esta fase en la *Figura 3*.

Crear un entorno para el entrenamiento del modelo
Diseñar la forma del tensor
Diseñar la arquitectura de capas del modelo
Optimizar hiperparámetros con Optuna
Visualizar datos de entrenamiento
Subir código a Git y keypoints a OneDrive
Cargar el dataset en el entorno
Normalizar tensores
Validar tensores normalizados
Crear un DataLoader personalizado
Crear RanzomZoom
Validar RanzomZoom
Crear HorizontalFlip
Validar HorizontalFlip
Crear desplazamiento X
Validar desplazamiento X
Crear desplazamiento Y
Validar desplazamiento Y

Figura 3 Historias de usuario segunda fase

2.2.3 Fase de Desarrollo y Entrenamiento del Modelo

En la última fase del proyecto, tras crear el código para cargar los datos y la creación del DataLoader, se procedió a diseñar las funciones necesarias para entrenar el modelo utilizando Optuna, una herramienta para la optimización de hiperparámetros, que permitió explorar y ajustar automáticamente los parámetros críticos del modelo.

Con las funciones de entrenamiento optimizadas, se llevó a cabo el proceso de entrenamiento inicial del modelo. Durante este proceso, se ejecutaron múltiples iteraciones para buscar la mejor combinación de hiperparámetros, que produjeron el mayor rendimiento del modelo según las métricas de evaluación definidas.

Una vez obtenidos estos parámetros, se validó el proceso de entrenamiento utilizando TensorBoard. Esta herramienta permitió monitorear en tiempo real las métricas de entrenamiento, como la pérdida y la precisión, lo cual facilitó la detección de posibles problemas y ajustes necesarios para mejorar el rendimiento del modelo, hasta la estabilización de las métricas de rendimiento y alcanzar la convergencia adecuada.

Las historias de usuario planteadas para la última parte del proyecto son las siguientes, y se pueden ver en la *Figura 4*.

Crear un diseño de modelo
Adaptar código para usar Optuna
Entrenar el modelo con Optuna
Determinar los mejores hiperparámetros
Validar el entrenamiento con TensorBoard
Crear el proceso de entrenamiento final
Entrenar el modelo final
Analizar resultados

Figura 4 Historias de usuario tercera fase

Después del entrenamiento final, se realizó una evaluación exhaustiva de los resultados obtenidos documentando e identificando posibles áreas de mejora, para futuras referencias y trabajos relacionados.

Capítulo 3 Materiales y métodos

3.1 Preprocesamiento de Datos

Durante la etapa previa al procesado de datos, se ha empleado un conjunto de aproximadamente 30.000 vídeos de combates de esgrima, previamente categorizados por Scott (Shalom) Dubinsky, disponibles en esta web [3]. De estos, se seleccionaron 7.694 vídeos en los que el ganador del punto es el tirador de la izquierda y 7.694 en los que es el tirador de la derecha. La selección se realizó en base a la calidad, duración y claridad de los vídeos, descartando aquellos que eran demasiado cortos o largos.

Para analizar estos vídeos de manera efectiva, la extracción de poses se convierte en una técnica fundamental en el aprendizaje automático. Esta técnica permite simplificar la información contenida en los vídeos y centrarse en los aspectos más relevantes del movimiento de los esgrimistas. Usar poses en el contexto del análisis de vídeo es crucial porque permite reducir la complejidad de las imágenes en movimiento a una serie de posturas clave que reflejan la posición y orientación del esgrimista en momentos específicos del combate. Esto no solo facilita la comparación y el análisis de los movimientos entre diferentes esgrimistas, sino que también mejora la precisión de los modelos de aprendizaje automático al proporcionar datos más claros y menos ruidosos. La *Figura 5* ilustra cómo se define una pose en este contexto.

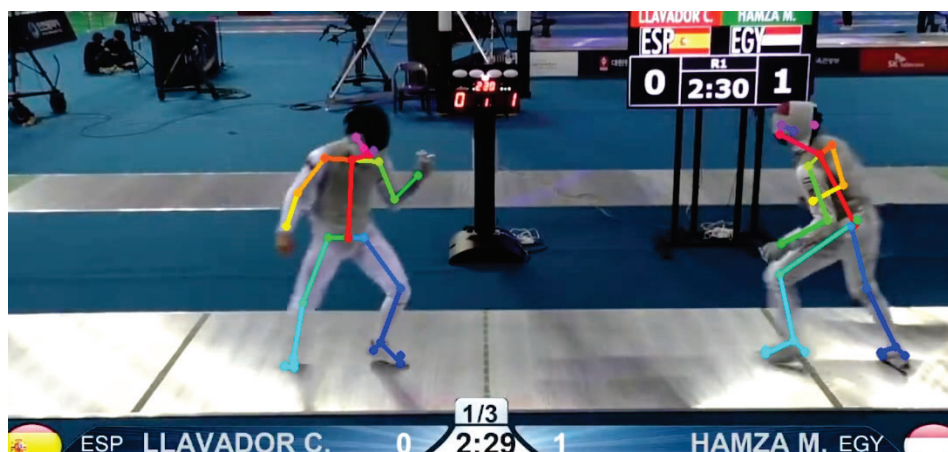


Figura 5 Identificación de poses en un fotograma

Para gestionar de manera eficiente los 30.000 vídeos utilizados, se han procesado sólo 1 de cada 10 fotogramas. Un fotograma, en el contexto del procesamiento de vídeo, es cada una de las imágenes individuales que componen el vídeo. Cada fotograma captura una instantánea del movimiento en un momento específico, y el análisis de estos es fundamental para comprender la dinámica temporal de los eventos, así como los movimientos de los tiradores en un combate. Como se puede ver en la *Figura 6*, que muestra la distribución de pares de poses detectadas en cada vídeo, la mayoría de los vídeos tienen entre 40 y 100 pares de poses.

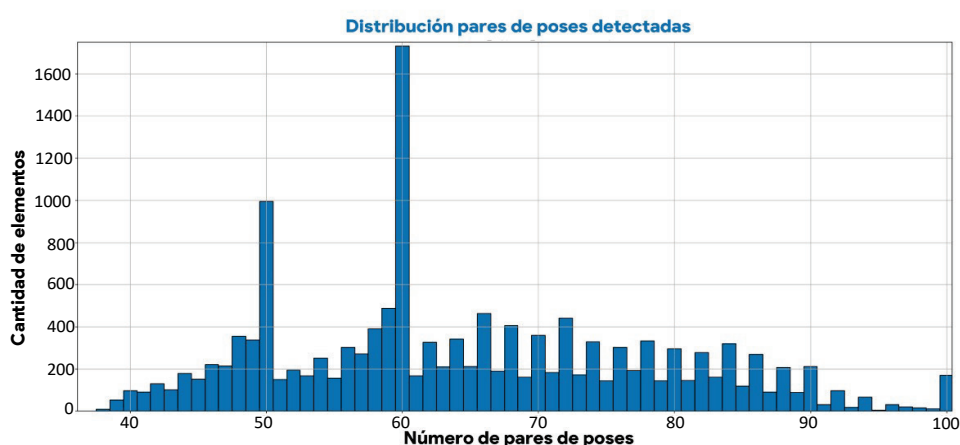


Figura 6 Distribución de pares de poses detectados

Aunque el proyecto se basa principalmente en las poses de los tiradores y no en la posición de los floretes (espadas), esta decisión se justifica porque no en todos los vídeos, ni en todos los fotogramas del vídeo, se ve el florete con claridad. Además, seguir la posición del florete sería mucho más costoso computacionalmente. Por tanto, analizar las poses de los tiradores es más eficiente y suficiente para interpretar las acciones realizadas y determinar cuál esgrimista se lleva el tocado, ya que las poses reflejan indirectamente la intención y el movimiento del esgrimista, permitiendo inferir acciones como los tocados.

Esta técnica ha sido ampliamente utilizada en estudios de reconocimiento de acciones y análisis de poses, para mejorar la técnica de los deportistas. Por ejemplo, el estudio de Chan-Yang Ju demuestra que el análisis de poses puede mejorar significativamente el rendimiento en el golf, permitiendo a los jugadores ajustar y perfeccionar sus técnicas a partir de la evaluación detallada de sus movimientos [9].

3.2 Redes Neuronales

Una red neuronal es una estructura computacional inspirada en el cerebro humano, diseñada para reconocer patrones en los datos. Estas redes consisten en capas de nodos o "neuronas", donde cada nodo está conectado a los nodos de la siguiente capa. La primera capa recibe la entrada (por ejemplo, imágenes o secuencias de datos), las capas ocultas procesan esta información, y la última capa produce la salida (por ejemplo, una clasificación o una predicción). Las redes neuronales aprenden ajustando los pesos de las conexiones entre nodos mediante un proceso de entrenamiento, donde se minimiza el error obtenido progresivamente usando técnicas de optimización como el descenso de gradiente. Este aprendizaje permite a la red capturar patrones complejos en los datos [10].

En este proyecto, se utilizan varios tipos de redes neuronales y técnicas avanzadas para analizar los vídeos de combates de esgrima. A continuación, se detallan las técnicas y arquitecturas empleadas.

3.2.1 Capas y arquitecturas habituales

3.2.1.1 Redes Neuronales Recurrentes (RNN) y LSTM

Las redes neuronales recurrentes (RNN, por sus siglas en inglés) son adecuadas para procesar datos secuenciales, como vídeos. Estas redes tienen una arquitectura que les permite recordar información de entradas anteriores, lo cual es esencial para datos que tienen un orden temporal, a diferencia de las redes tradicionales que pueden tener dificultades para recordar información relevante de secuencias largas debido al problema del desvanecimiento del gradiente, donde los gradientes necesarios para ajustar los pesos se vuelven muy pequeños.

Para superar este problema, se utilizan las Long Short-Term Memory (LSTM), una variante de las RNN que puede aprender dependencias a largo plazo en secuencias de datos. Las LSTM están diseñadas para retener información relevante a lo largo de secuencias más largas, lo cual es crucial en la comprensión de la evolución de las posiciones y movimientos de los esgrimistas durante el combate. Las capas LSTM en el modelo capturan la secuencia temporal de las acciones, permitiendo al modelo entender cómo evolucionan a lo largo del tiempo [11].

3.2.1.2 Capas Densas

Las capas densas, o completamente conectadas, son capas en las cuales cada nodo está conectado a todos los nodos de la capa anterior y de la capa siguiente. Estas capas son fundamentales en la toma de decisiones del modelo. Después de que las capas LSTM procesan la secuencia de datos, las capas densas toman las características extraídas y realizan la clasificación final para predecir el resultado del combate.

3.2.2 Funciones de Activación

Las funciones de activación introducen no linealidad en el modelo, permitiendo representar relaciones más complejas en los datos:

3.2.2.1 Leaky ReLU

La activación Leaky ReLU es una variante de la función ReLU que permite que los valores negativos pasen a través de la función en lugar de asignarles un valor de cero. Esto mitiga el problema de "muertes" de neuronas durante el entrenamiento, acelera el proceso de entrenamiento y evita problemas de desvanecimiento del gradiente.

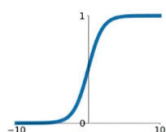
3.2.2.2 Función Sigmoide

La función sigmoide se utiliza en la capa de salida para predecir la probabilidad de que una instancia pertenezca a una clase específica. Esta función comprime los valores de entrada en un rango entre 0 y 1, interpretándolos como probabilidades. La parte izquierda de la *Figura 7* muestra la función sigmoide.

Funciones de activación

Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



Leaky ReLU

$$\max(0.1x, x)$$

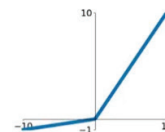


Figura 7 Funciones de activación [S. Rahman]

3.2.3 Técnicas de Optimización y Regularización

Para garantizar un rendimiento óptimo del modelo y evitar el sobreajuste, se aplicaron diversas técnicas como las que se detallan a continuación.

3.2.3.1 DataLoader

El DataLoader es una herramienta que facilita la carga y procesamiento de datos de manera eficiente. En proyectos de aprendizaje automático, a menudo se trabaja con grandes volúmenes de datos que deben ser procesados en lotes para que el entrenamiento sea más manejable y rápido. El DataLoader organiza los datos en lotes y los preprocesa, permitiendo que se alimenten al modelo de manera continua y eficiente durante el entrenamiento y la validación. En este

proyecto, se ha creado un DataLoader personalizado que maneja los datos de entrenamiento y validación, asegurando que estén listos para ser utilizados por el modelo de forma óptima [12].

3.2.3.2 Data Augmentation

Data Augmentation es una técnica utilizada para aumentar la cantidad y diversidad de los datos de entrenamiento mediante la generación de variantes artificiales de los datos originales. Esto se logra aplicando transformaciones como rotaciones, traslaciones y cambios de escala a los datos de entrada. Estas transformaciones crean nuevas versiones de los datos existentes, lo que ayuda al modelo a generalizar mejor y ser más robusto frente a variaciones en los datos de entrada. En el código del proyecto, se utiliza una función para crear generadores de datos que aplican Data Augmentation durante el entrenamiento, aumentando así la diversidad de los datos de entrada y mejorando el rendimiento del modelo.

3.2.3.3 Optimización de Hiperparámetros con Optuna

La optimización de hiperparámetros es un proceso crucial en el diseño de modelos de aprendizaje automático, ya que los hiperparámetros influyen significativamente en el rendimiento del modelo. Optuna es una biblioteca que facilita la búsqueda de hiperparámetros óptimos mediante estrategias de búsqueda inteligente, como la optimización bayesiana, que reducen el número de combinaciones a probar y aceleran el proceso de optimización. En este proyecto, se utiliza Optuna para explorar diferentes combinaciones de hiperparámetros, como la tasa de aprendizaje, el tamaño de las capas y el porcentaje de transformación de los datos, encontrando la configuración que maximiza el rendimiento del modelo [13].

3.2.3.4 Early Stopping

Early Stopping es una técnica que detiene el entrenamiento de la red neuronal cuando el rendimiento del modelo deja de mejorar significativamente en el conjunto de validación. Esto ayuda a prevenir el sobreajuste y reduce el tiempo de

entrenamiento. En el código del proyecto, se utiliza un callback de Early Stopping que evalúa la precisión y la pérdida del modelo, deteniendo el entrenamiento cuando no hay mejoras después de un número determinado de épocas. Este callback considera varias métricas a la vez para determinar el momento óptimo para detener el entrenamiento.

3.2.3.5 Regularización

La regularización es una técnica utilizada para reducir el riesgo de sobreajuste en modelos de aprendizaje automático. En este proyecto, se utiliza la regularización L2, que penaliza los valores extremos de los pesos del modelo durante el entrenamiento. Esta penalización se suma al error de predicción, forzando al modelo a mantener los pesos más pequeños, lo que ayuda a prevenir la complejidad excesiva y promueve una mayor generalización del modelo [14] Srivastava, N., et al. (2014). Dropout: A simple way to prevent neural networks from overfitting. Journal of Machine Learning Research.[14].

3.2.4 División de Conjuntos de Datos

La división del conjunto de datos en conjuntos de entrenamiento, validación y prueba es fundamental para un entrenamiento efectivo del modelo. Esta división garantiza que el modelo se entrene en un conjunto de datos separado de los datos que se utilizarán para evaluar su rendimiento.

El conjunto de entrenamiento se utiliza para ajustar los pesos del modelo durante el proceso de entrenamiento. El conjunto de validación se utiliza para ajustar los hiperparámetros del modelo y prevenir el sobreajuste. Finalmente, el conjunto de prueba se reserva para evaluar el rendimiento final del modelo en datos no vistos.

Esta división es esencial para evaluar si el modelo es capaz de procesar bien con datos nuevos y no vistos durante el entrenamiento.

3.2.5 Optimizadores

Los optimizadores son algoritmos utilizados para ajustar los pesos del modelo durante el entrenamiento. En este proyecto, se evalúan varios optimizadores, cada uno con sus características:

3.2.5.1 Adam (*Adaptive Moment Estimation*)

Adam es un optimizador que combina las ventajas de los algoritmos AdaGrad y RMSProp. Este optimizador es ampliamente utilizado debido a su eficiencia y bajo requerimiento de memoria. Adam adapta la tasa de aprendizaje para cada parámetro individualmente, lo que lo hace adecuado para una variedad de problemas de optimización.

3.2.5.2 RMSprop (*Root Mean Square Propagation*)

RMSprop es un optimizador que ajusta la tasa de aprendizaje para cada parámetro del modelo. En comparación con otros optimizadores, RMSprop es particularmente útil para problemas con grandes cantidades de datos. Este optimizador adapta de forma automática la tasa de aprendizaje, lo que lo hace robusto y eficaz para optimizar modelos en conjuntos de datos extensos.

3.2.5.3 SGD (*Stochastic Gradient Descent*)

SGD (Stochastic Gradient Descent) es un optimizador clásico utilizado para actualizar los pesos del modelo en cada iteración utilizando un subconjunto aleatorio de datos de entrenamiento. Aunque SGD es simple, es efectivo para problemas de optimización en los que los datos son abundantes.

3.2.5.4 Adagrad (*Adaptive Gradient Algorithm*)

Adagrad es un optimizador que adapta la tasa de aprendizaje a los parámetros del modelo. Este optimizador realiza actualizaciones más grandes para parámetros poco frecuentes y actualizaciones más pequeñas para parámetros frecuentes. Adagrad es útil en problemas donde los datos tienen diferentes escalas y cuando se necesita una adaptación rápida de la tasa de aprendizaje para cada parámetro.

3.2.5.5 *Adadelata*

Adadelata es una extensión de Adagrad que busca reducir su decrecimiento agresivo de la tasa de aprendizaje. Este optimizador adapta la tasa de aprendizaje en función de las actualizaciones anteriores de los pesos del modelo, lo que lo hace más robusto en problemas de optimización a largo plazo y evita la necesidad de ajustar manualmente la tasa de aprendizaje. Adadelata es útil en problemas donde se necesita una convergencia estable y rápida del modelo.

3.2.6 Métricas de Evaluación

En el proceso de entrenamiento y evaluación de modelos de aprendizaje automático, las métricas de evaluación desempeñan un papel fundamental. Son herramientas esenciales para comprender y medir el rendimiento de un modelo.

Estas métricas son cruciales tanto para el proceso de optimización de hiperparámetros con Optuna como para la técnica de detención temprana (Early Stopping) durante el entrenamiento del modelo, unas de las más utilizadas y presentes en el proyecto son:

3.2.6.1 *Accuracy*

El Accuracy es la precisión del modelo, que mide la proporción de predicciones correctas e incorrectas. Esta métrica es fundamental para evaluar el rendimiento general del modelo en la clasificación de las clases.

3.2.6.2 *AUC (Area Under the ROC Curve)*

El AUC mide la capacidad del modelo para distinguir entre clases. Representa el área bajo la curva ROC (Receiver Operating Characteristic) y es una medida de la capacidad de discriminación del modelo. Un AUC más alto indica un mejor rendimiento del modelo en la clasificación.

3.2.6.3 F1 Score

El F1 Score es la métrica F1, que es la media armónica de la precisión y el recall. Proporciona una medida equilibrada del rendimiento del modelo, especialmente en conjuntos de datos desbalanceados. El F1 Score es útil cuando las clases tienen diferentes distribuciones y es una métrica robusta para evaluar el rendimiento del modelo en clasificaciones binarias y multiclase.

Capítulo 4 Diseño e implementación

4.1 Recursos del proyecto

Para facilitar la replicación y revisión del proyecto, se ha subido todo el código desarrollado a un repositorio en GitHub [1] y se ha creado un notebook en Google Colab [2] que contiene el flujo completo del proyecto. Estos recursos permiten a los interesados explorar el código, realizar modificaciones y ejecutar el código de manera sencilla.

4.2 Preprocesamiento de datos

El preprocesamiento de datos es una etapa crucial en el desarrollo de modelos de aprendizaje automático, donde se preparan y organizan los datos para su posterior análisis y entrenamiento. En el contexto de este proyecto, el preprocesamiento de datos implica una serie de pasos fundamentales que se describen a continuación:

4.2.1 Arranque y creación del dataset

En primer lugar, para la creación del dataset, se excluyeron los vídeos con una duración inferior a cinco segundos, ya que estos no aportarían valor al proceso de entrenamiento, siendo mayormente fragmentos irrelevantes o cortes sin contexto.

Una vez recopilados los vídeos pertinentes, se procedió a segmentarlos y categorizarlos según el resultado del combate, identificando a qué tirador correspondía el punto. Esta información, presente en una lista adjunta al dataset, permitió crear dos grupos equilibrados de vídeos, uno para los tocados de la izquierda y otro para los de la derecha.

La correcta categorización de los vídeos aseguró que el conjunto de datos estuviera etiquetado de manera precisa y balanceada, lo cual es esencial para el entrenamiento efectivo del modelo, proporcionando a su vez una información clave sobre las acciones que llevaron al éxito a cada tirador.

4.2.2 Obtención de las poses

A continuación, y una vez categorizados los vídeos se procedió a obtener las poses de cada vídeo con OpenPose. Cada pose está compuesta por un conjunto de puntos clave, donde cada punto está representado por tres coordenadas: x, y, y la confianza asociada a esa detección. Así, si consideramos dos tiradores en un vídeo, cada uno con sus 25 puntos clave, obtendremos un total de 150 números que componen la representación de las poses de ambos esgrimistas cómo se puede ver en la Figura 8.

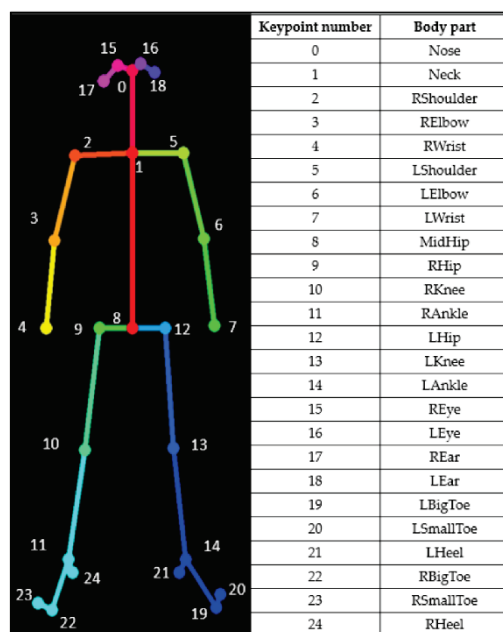


Figura 8 Representación de puntos clave [7]

Estos números proporcionan información detallada sobre la posición y orientación de cada parte del cuerpo de los esgrimistas en un momento dado del combate. Por ejemplo, las coordenadas x e y indican la posición en el plano, mientras que la

coordenada ultima indica la confianza en la detección de ese punto específico. Al tener esta información para cada fotograma del vídeo, se obtiene una secuencia temporal de poses que permite seguir el movimiento y la dinámica de los esgrimistas a lo largo del combate [15].

4.2.3 Filtrado de poses no válidas

Seguidamente se llevó a cabo un proceso de filtrado de los vídeos para descartar aquellos que no cumplieran con los estándares de calidad y precisión necesarios para el entrenamiento del modelo. Los criterios de eliminación se establecieron en base a dos aspectos principales.

En primer lugar, se descartaron los vídeos que presentaban poses incompletas de los esgrimistas. Esto incluía situaciones como fotogramas con información faltante sobre las poses debido a cortes en los vídeos o problemas en el seguimiento de los tiradores, donde estos salían de escena o la herramienta OpenPose no lograba detectar correctamente sus poses. Esta medida garantizó que se utilizara únicamente información completa y coherente para el entrenamiento del modelo.

Por otro lado, se eliminaron las poses o elementos que podían introducir ruido en el modelo y afectar a su precisión. Esto comprendía la presencia de personas u objetos ajenos al combate de esgrima, como el público, árbitros, entrenadores u otros tiradores de fondo. Incluso en ocasiones, elementos como el marcador podían ser detectados erróneamente como parte de las poses de los esgrimistas.

Para abordar este problema, se implementó un algoritmo que analizaba la posición y el tamaño de los puntos clave en cada fotograma para identificar y eliminar las poses intrusas con un alto grado de precisión. Este algoritmo se diseñó teniendo en cuenta que los tiradores, al estar normalmente en primer plano, suelen tener poses con mayor dispersión de puntos o un mayor número de puntos detectados. El script utilizado para este proceso se encuentra disponible en el repositorio de GitHub adjunto con la memoria del proyecto [5].

Este script se compone de una función principal, *filter_poses*, que recibe los datos de un fotograma y filtra las poses no válidas. Se eliminan las poses que no cumplen con un número mínimo de puntos clave y cuya dispersión media de puntos está por debajo de un umbral establecido. Esto se logra calculando la dispersión estándar de los puntos clave y verificando si esta supera el umbral definido.

4.2.4 Formateo y ordenación de las poses

Para preparar las poses obtenidas, se definieron las dimensiones del tensor de datos resultante. La primera dimensión corresponde al número de muestras, es decir, la cantidad de vídeos en el conjunto de datos. La segunda dimensión representa el número de fotogramas de los cuales se ha obtenido una pose. Es importante destacar que este número no refleja directamente la cantidad de fotogramas en los vídeos originales, ya que las poses fueron extraídas cada 10 fotogramas reales. En otras palabras, los vídeos originales contienen diez veces más fotogramas de los utilizados para obtener las poses.

Para mantener la consistencia en las poses, todas las secuencias deben tener la misma duración. Por lo tanto, se implementó un método de "zero padding" para los casos en los que faltaba información. Si un vídeo no contenía el número esperado de poses (calculado dividiendo el número total de fotogramas entre 10), las poses faltantes se completaban con ceros hasta alcanzar la dimensión correspondiente al tamaño máximo de 100. La tercera dimensión del tensor corresponde al número de personas en la escena, que en este caso es fijo en 2. Finalmente, la cuarta dimensión indica el número de puntos clave por persona. Inicialmente, se consideraban las coordenadas x, y, y la confianza para cada punto. Sin embargo, la confianza no aportaba información relevante al entrenamiento y dificultaba el aprendizaje del modelo, por lo que fue eliminada. De esta manera, ahora solo se utilizan las coordenadas x e y, lo que reduce el número de coordenadas clave por pose de 75 a 50. Si en algún fotograma no se detectaban los 25 puntos clave para ambas personas, se rellenaban las coordenadas x e y de esos puntos con ceros para mantener la coherencia en las dimensiones del tensor.

Además, se implementó un proceso para ordenar las poses de manera que la primera pose de las dos siempre sea la que está más a la izquierda y la segunda pose sea la que está a la derecha. Esta medida fue esencial debido a que OpenPose ordena de forma aleatoria las poses detectadas. Además, el proceso de filtrado para descartar poses no válidas también puede alterar el orden original de las poses. Este orden es crucial para identificar qué pose corresponde a cada tirador, por lo que la reorganización se convirtió en un paso necesario para mantener la coherencia en la interpretación de los datos. Para determinar el orden de las poses, se calculó la media de las coordenadas x de los puntos clave de cada persona. Si la media de la persona a la izquierda era menor que la de la persona a la derecha, se mantenían las poses en su orden original; de lo contrario, se invertían.

4.3 División de datasets usados para el entrenamiento.

La división de los conjuntos de datos se ha realizado para evitar que el modelo memorice los datos de validación durante el entrenamiento, asegurando así una evaluación justa. Además, se ha creado una división adicional de test para probar el entrenamiento y el refinamiento final del modelo, proporcionando una evaluación más precisa del modelo. Esta estrategia busca garantizar la independencia entre las distintas fases del proceso de entrenamiento, validación y evaluación. Asimismo, se han dividido los datos específicamente para su uso con Optuna, y para la implementación de un Early stopping.

4.4 Creación de Dataloader

En el contexto de este proyecto, se ha creado un DataLoader específico para manejar el conjunto de datos de vídeos de combates de esgrima.

Además de cargar los datos de manera incremental, el DataLoader también se ha configurado para implementar técnicas de data augmentation, una estrategia

crucial en el aprendizaje automático, especialmente cuando se trabaja con conjuntos de datos limitados o desbalanceados.

En este proyecto, se han aplicado diversas transformaciones durante la carga de datos, como el zoom in y zoom out, que modifican el tamaño de las poses de los esgrimistas en los vídeos. Estas transformaciones se realizan mediante operaciones con tensores, lo que permite un procesamiento más rápido y eficiente. Para implementar el zoom, se obtienen las coordenadas mínimas y máximas de los puntos de la pose, se calcula un factor de zoom aleatorio dentro de un rango permitido y se ajustan las coordenadas de los puntos en consecuencia.

Otra transformación aplicada es el flip horizontal, que invierte la posición de los esgrimistas, simulando que el vídeo fue tomado desde el otro lado de la pista. Para realizar esta transformación, se obtienen las coordenadas x de cada punto de la pose y se invierten, cambiando también el orden de las poses y ajustando las etiquetas de entrenamiento [18].

También se ha utilizado una transformación de translación en los ejes x e y, desplazando las poses de los esgrimistas en el espacio e introduciendo variaciones en su posición en relación con el vídeo original. Esta transformación se implementa calculando un desplazamiento aleatorio dentro de un rango permitido y aplicándolo a las coordenadas de los puntos de la pose.

Estas técnicas de data augmentation permiten enriquecer el conjunto de datos de entrenamiento, proporcionando al modelo una mayor diversidad de ejemplos para aprender, mejorando su capacidad de generalización a situaciones reales de combate de esgrima y evitando el overfitting.

La implementación de estas transformaciones se ha realizado utilizando TensorFlow, aprovechando las capacidades de procesamiento en GPU y la precisión mixta para acelerar el entrenamiento. Esto no solo mejora la eficiencia

del procesamiento de datos, sino que también permite manejar conjuntos de datos más grandes y complejos de manera más efectiva [16].

4.5 Diseño de arquitectura

La función de pérdida Binary Cross-Entropy (BCE) se seleccionó como criterio de optimización para abordar un problema de clasificación binaria en el análisis de secuencias de video.

La BCE se destaca como una elección óptima para este escenario debido a su capacidad para manejar problemas con dos clases mutuamente excluyentes. Esta función opera eficientemente con salidas probabilísticas, lo cual es fundamental para estimar la victoria de cada tirador. Durante el proceso de entrenamiento, la BCE aplica una penalización logarítmica a las predicciones erróneas, proporcionando así un gradiente significativo para el ajuste de los parámetros del modelo, cómo se puede ver en la Figura 9.

$$-\frac{1}{N} \sum_{i=1}^N y_i \cdot \log(p(y_i)) + (1 - y_i) \cdot \log(1 - p(y_i))$$

Figura 9 Formula Binary Cross-Entropy [6]

La minimización de la BCE durante el entrenamiento conduce a una optimización de los pesos de la red neuronal. Este proceso iterativo refina la capacidad del modelo para discernir los patrones visuales asociados con la victoria de cada tirador. Como resultado, se obtiene un clasificador robusto capaz de analizar nuevos fragmentos de video y predecir con precisión si el tirador de la derecha o de la izquierda emerge como ganador en una acción específica [19].

Cómo se puede ver en la *Figura 10 Modelo de capas*, el modelo empleado es una red neuronal recurrente (RNN) bidireccional con capas LSTM para capturar

patrones temporales. El modelo incluye una capa de masking que ignora el relleno de ceros, asegurando que estos valores no afecten el aprendizaje. Los números al lado de las capas indican las dimensiones de salida de cada capa. Por ejemplo, "LSTM (64)" significa que la capa LSTM tiene 64 unidades. Se utiliza dropout para regularizar el modelo y prevenir el sobreajuste, y una capa densa final con activación sigmoide para la clasificación binaria [14].

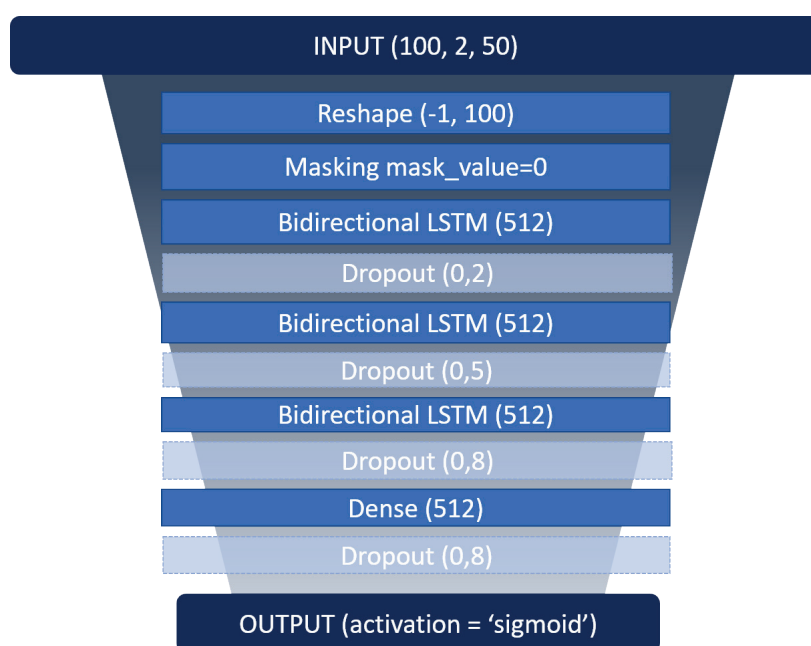


Figura 10 Modelo de capas

4.6 Entrenamiento con Optuna

Una vez configurado el DataLoader para cargar los datos y aplicar técnicas de data augmentation, se procedió a entrenar el modelo utilizando la biblioteca Optuna para realizar un ajuste automático de hiperparámetros. Esta herramienta permitió explorar una amplia gama de configuraciones del modelo y encontrar aquellas que mejor se adaptan al problema específico de análisis de vídeos de combates de esgrima.

El proceso de entrenamiento con Optuna consistió en realizar una serie de intentos, donde cada intento evaluaba un conjunto diferente de hiperparámetros.

Esto incluía parámetros como la tasa de aprendizaje, el número de unidades en las capas LSTM y densas, y el dropout.

Por ejemplo, en el código proporcionado, se define una función objetivo que toma como argumento un objeto "trial" de Optuna. Dentro de esta función, se definen los hiperparámetros que se desean optimizar, como la tasa de aprendizaje, el número de unidades LSTM, el número de unidades densas y la tasa de dropout, como se puede ver en la *Tabla 1*.

Después, se creó un diccionario con estos parámetros que se utilizaron para construir el modelo [17].

RANGOS HIPERPARÁMETROS	
learning_rate	(0.00001, 0.001)
lstm_units	[128,256,512]
dropout_rate_1	(0.05, 0.95)
dropout_rate_2	(0.05, 0.95)
dropout_rate_3	(0.05, 0.95)
dropout_rate_4	(0.05, 0.95)
dense_units	[128,256,512]
activation	['sigmoid','leaky_relu']
optimizer	['Adam','SGD','RMSprop ','Adadelta','Adagrad ']
regularization_type	['l1','l2']
regularization	(1e-06, 1e-05)
transformation_prob	(0.0, 1.0)

Tabla 1 Rangos hiperparámetros

4.7 Entrenamiento final

Tras el estudio de Optuna para encontrar los mejores hiperparámetros, se procedió a realizar el entrenamiento final del modelo. En este paso, se combinó el conjunto de datos de entrenamiento y validación para formar un conjunto de datos más amplio y completo. Luego, se utilizó este conjunto de datos combinado para entrenar y validar el modelo, asegurando que se aprovechara al máximo la información disponible.

Una vez completado el entrenamiento final, se llevó a cabo la evaluación final utilizando un conjunto de datos que no había sido utilizado anteriormente ni para el entrenamiento ni para la validación.

Capítulo 5 Resultados

5.1 Resultados de script de filtrado

Una parte fundamental del proyecto consistió en verificar que OpenPose capturaba con precisión las posturas de los esgrimistas, incluso en movimientos rápidos y complejos, como se muestra en la *Figura 11*.



Figura 11 Extracción de poses de un vídeo con OpenPose

De este modo, se identificó que además de las poses de los tiradores del combate analizado, OpenPose detectaba otros elementos no útiles para el propósito de vídeo arbitraje. Utilizando el script de filtrado presentado en la sección 4.2.3, se eliminaron dichos elementos no útiles, tal y como se muestra en la *Figura 12*, donde se observa la detección errónea de una pose en un videomarcador, y su eliminación aplicando el script.

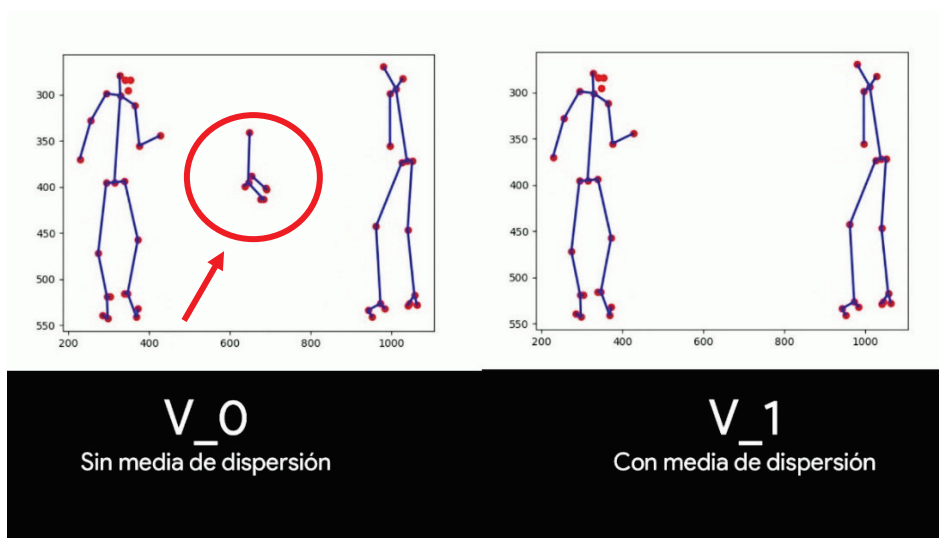


Figura 12 Detección errónea de una pose en videomarcador

Además, OpenPose a veces detecta otras figuras humanas en el fondo del vídeo que no son de interés. Estas poses también se eliminan descartando aquellas con poca dispersión, como se muestra en la *Figura 13*.

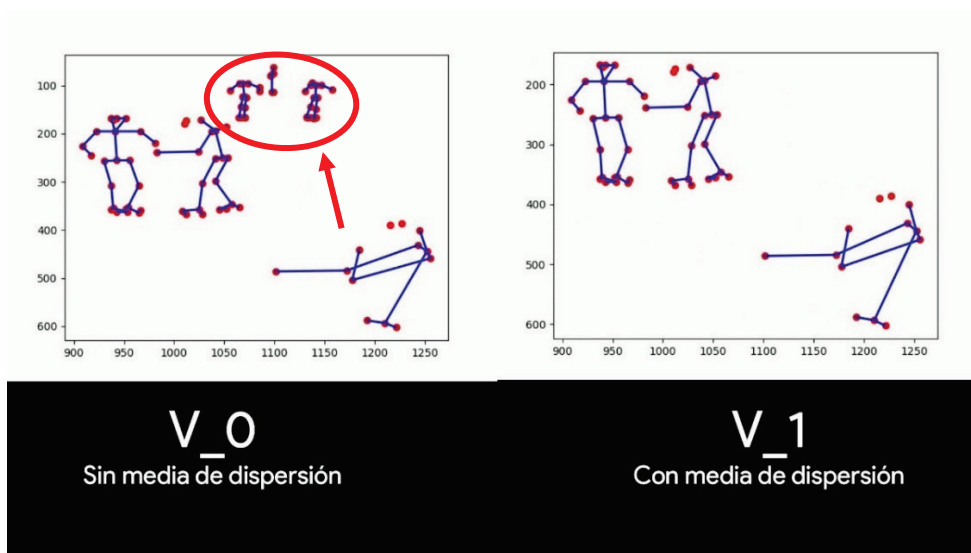


Figura 13 Detección errónea de poses de otros esgrimistas del fondo

En la *Figura 14*, se muestra la detección errónea del árbitro. Aunque está en primer plano, su dispersión de puntos es similar a la de los esgrimistas, por lo que se aplicó una segunda estrategia que elimina poses con pocos puntos, eliminando así al árbitro de la detección.

Después de aplicar estas dos estrategias, (dispersión y número de puntos), se confirmó que las poses incorrectamente detectadas por OpenPose no se incluían ya en los resultados finales, confirmando así el éxito de las dos estrategias utilizadas.

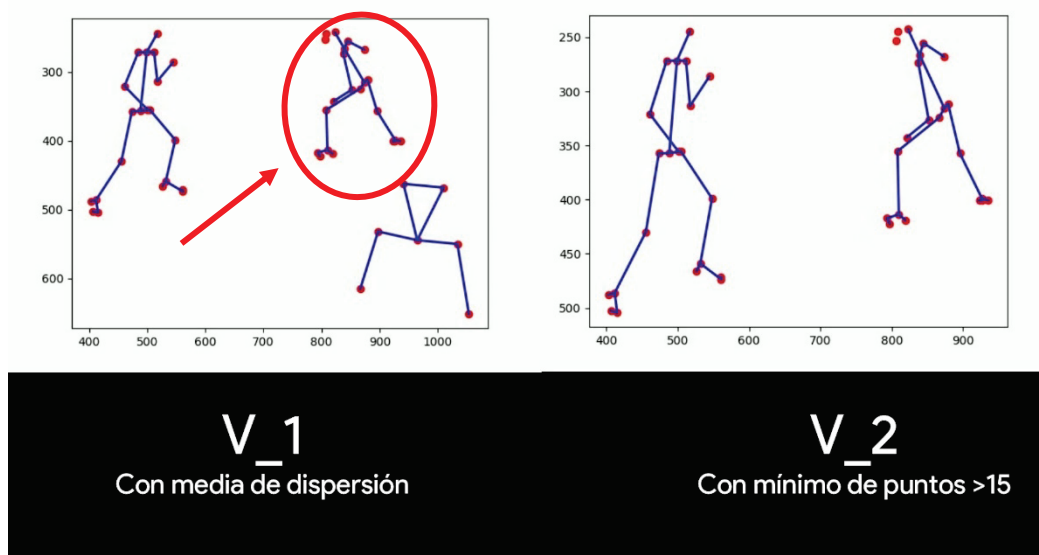


Figura 14 Detección errónea del arbitro

5.2 Resultados de las transformaciones

Para asegurar la validez de las transformaciones de las poses, se realizaron tanto validaciones visuales como numéricas.

Durante la validación visual, se inspeccionaron manualmente las poses generadas para asegurarse de que representaban fielmente las posturas reales de los esgrimistas. Se seleccionaron varias poses al azar y se compararon con los fotogramas originales del vídeo. Esta inspección confirmó que las poses mantenían la integridad de los movimientos y posturas originales, validando la precisión de la transformación. Como se puede ver en la *Figura 15*, donde a la izquierda se ven las poses originales y a la derecha las transformadas.

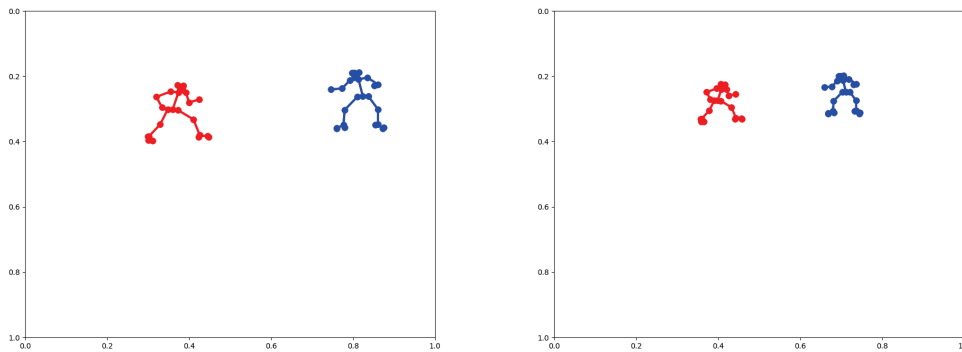


Figura 15 Transformación de zoom

Otro ejemplo de validación visual sobre el mismo vídeo es el que muestra la *Figura 16*, donde a la izquierda se ven las poses originales y a la derecha un desplazamiento de la secuencia de poses en el eje y.

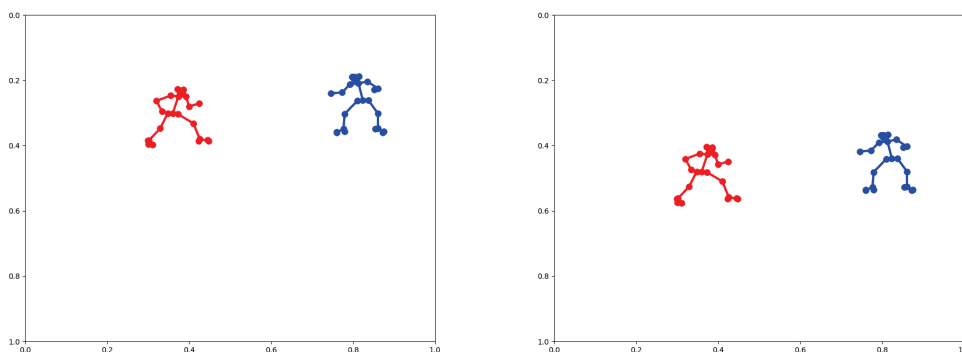


Figura 16 Transformación desplazamiento eje Y

Durante la validación numérica se implementaron funciones matemáticas para verificar que las coordenadas de las poses transformadas se mantuvieran dentro del intervalo $[0, 1]$. Estas funciones compararon las coordenadas generadas con los límites establecidos, asegurando que no hubiera desbordamientos o errores en las transformaciones. Los resultados numéricos mostraron que todas las poses transformadas se encontraban dentro del rango esperado. Además, se calculó el tiempo necesario para aplicar las transformaciones, comprobando también la eficiencia de estas.

En la *Tabla 2* se pueden ver los resultados de la validación de las transformaciones y el tiempo de ejecución. El porcentaje de normalización del 400% se debe a que se aplicaron cuatro transformaciones distintas a cada tensor. Esto implica que cada tensor fue normalizado cuatro veces, una por cada transformación, lo que explica el valor elevado del porcentaje.

Dataset	Tensores totales	Tensores normalizados	Porcentaje normalizado	Tiempo ejecución
Entrenamiento	19558	19558	400%	293,80 segundos
Validación	8384	8384	400%	133,65 segundos

Tabla 2 Validación transformaciones y tiempo de ejecución

5.3 Resultados de entrenamiento

La *Tabla 3* muestra las métricas obtenidas en la evaluación del modelo, que incluyen la pérdida, precisión, área bajo la curva ROC y F1 Score.

Métrica	Valor
Pérdida (ValLoss)	0.6723
Precisión (ValAccuracy)	0.6093
AUC	0.6670
F1 Score	0.6653

Tabla 3 Resultados

Al analizar en profundidad las métricas de evaluación del modelo diseñado para arbitrar acciones de combates de esgrima, se observa que los resultados obtenidos no son satisfactorios, lo que indica limitaciones significativas en su capacidad de predicción. La precisión del modelo es del 60.93%.

Como se ha comentado anteriormente en el apartado 3.2.6, la precisión mide la proporción de verdaderos positivos sobre el total de predicciones positivas, y un 60.93% indica que aproximadamente 6 de cada 10 predicciones son correctas. Este valor sugiere que el modelo comete errores en casi el 40% de los casos, lo cual es un margen de error demasiado alto para la tarea crítica de arbitrar combates de esgrima, donde decisiones precisas son esenciales para determinar el ganador de cada asalto.

El área bajo la curva ROC (AUC) es de 0.6670, lo cual mide la capacidad del modelo para distinguir entre acciones positivas y negativas en los combates. Un AUC de 0.6670 indica que el modelo tiene una capacidad moderada para diferenciar entre las dos clases, siendo mejor que un modelo aleatorio, que tendría un AUC de 0.5. Aunque este valor es mejor que el azar, sigue siendo relativamente bajo, indicando que el modelo no es muy efectivo en la diferenciación entre clases.

El F1 Score del modelo es de 0.6653, el cual es la media armónica de la precisión y el recall, ofreciendo una medida balanceada entre ambos. Este valor sugiere que el modelo tiene un rendimiento aceptable en términos de precisión y recall, aunque aún está lejos de ser excelente, indicando que hay un margen considerable para mejorar tanto en la precisión como en el recall del modelo.

A pesar de la utilización de transformaciones de datos, los resultados no son satisfactorios, lo que sugiere la posibilidad de overfitting. El overfitting ocurre cuando el modelo se ajusta demasiado bien a los datos de entrenamiento, capturando patrones irrelevantes que no generalizan bien a nuevos asaltos de esgrima, como se puede ver en la *Figura 17*. En el caso del modelo de arbitraje de esgrima, esto podría significar que el modelo ha aprendido características específicas de los combates de entrenamiento, pero no es capaz de aplicar este conocimiento de manera efectiva a nuevos combates. Este fenómeno se ve exacerbado cuando se utilizan modelos demasiado complejos, con muchas características o parámetros, ya que estos modelos tienden a ajustarse en exceso a los datos de entrenamiento.

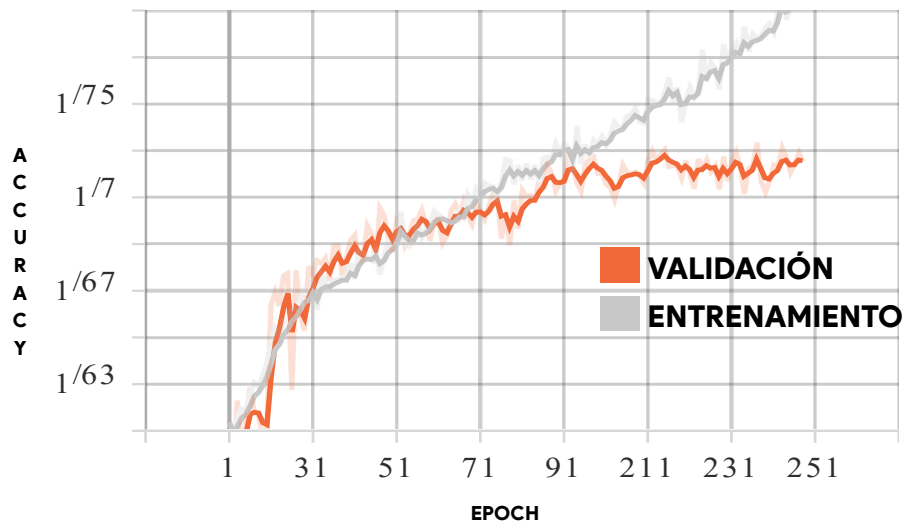


Figura 17 Gráfico Accuracy (BinaryCross-Entropy)

El modelo, al ser demasiado complejo, puede haber aprendido detalles específicos y ruidos propios de los datos de entrenamiento que no son representativos del verdadero comportamiento de las acciones en los combates de esgrima. Por ejemplo, podría estar identificando patrones de movimientos o señales que son únicos de ciertos combates de entrenamiento y no son generalizables a otros combates. Esto significa que, aunque el modelo parece funcionar bien con los datos de entrenamiento, su rendimiento se deteriora cuando se le presentan nuevos datos, lo cual es crítico en el contexto de arbitraje de esgrima, donde la capacidad de generalizar es fundamental para tomar decisiones justas y precisas en diferentes situaciones y con diferentes deportistas.

Capítulo 6 Conclusiones

6.1 Resumen del Trabajo

Este trabajo ha abordado el desafío de desarrollar un sistema de vídeo arbitraje asistido por IA para la esgrima, con el objetivo de mejorar la precisión y la consistencia en la toma de decisiones arbitrales. Para ello, se ha creado un modelo de aprendizaje profundo que utiliza redes neuronales recurrentes para analizar secuencias de poses de esgrimistas extraídas de vídeos de combates. Se ha puesto especial énfasis en el preprocesamiento de datos, la optimización de hiperparámetros y la aplicación de técnicas de aumento de datos para mejorar el rendimiento del modelo. A pesar de los esfuerzos realizados, los resultados obtenidos no han alcanzado el nivel de precisión deseado para su aplicación práctica en el arbitraje real.

6.2 Limitaciones y oportunidades futuras

Este trabajo ha buscado desarrollar un sistema capaz de asistir en el arbitraje de este deporte. A pesar de algunos aspectos prometedores, los resultados obtenidos no han alcanzado el nivel de eficacia esperado para su aplicación práctica. Este análisis explora las limitaciones actuales identificadas y las oportunidades futuras para mejorar el sistema.

6.2.1 Limitaciones Actuales

Una posible causa del rendimiento subóptimo del modelo es el tamaño limitado del conjunto de datos utilizado. Esta restricción podría haber afectado su capacidad de generalización, al igual que la calidad y variedad de los vídeos, que podrían haber sido insuficientes para un entrenamiento efectivo.

Si bien la detección de poses ofrece un gran potencial para capturar movimientos y posturas con precisión, su uso exclusivo en este proyecto ha demostrado ser una limitación. La falta de información sobre la posición del florete, omitida inicialmente por su complejidad computacional, ha dificultado la interpretación de las acciones y la determinación del ganador del punto.

Otro factor que podría haber influido en la calidad de los resultados es el proceso de validación y categorización de los vídeos. Dado que depende de la opinión de un número limitado de individuos, esto podría suponer un riesgo de sesgo y haber restringido la diversidad de perspectivas necesarias para una categorización robusta de las acciones.

Es importante señalar que estas limitaciones requieren una investigación más profunda para determinar su impacto real en los resultados obtenidos.

6.2.2 Oportunidades Futuras

Las limitaciones identificadas ofrecen claras oportunidades de mejora para futuras iteraciones del proyecto. Una prioridad fundamental es la ampliación significativa del conjunto de datos. Esto implica no solo aumentar la cantidad de vídeos disponibles, sino también asegurar una mayor diversidad y calidad en los mismos. Se propone la obtención de grabaciones desde múltiples ángulos, lo que permitiría capturar una gama más amplia de situaciones y mejorar la capacidad del modelo para generalizar.

Para abordar la problemática de la detección del florete, se sugiere la implementación de algoritmos específicos que combinen la detección de poses con técnicas avanzadas de reconocimiento de objetos. Esta aproximación híbrida podría mejorar significativamente la precisión en la identificación del arma y, por extensión, la interpretación de los tocos durante el combate.

Con el objetivo de mejorar la validación y categorización de los vídeos, se propone el desarrollo de una aplicación especializada. Esta plataforma permitiría a múltiples expertos, incluyendo árbitros y esgrimistas con experiencia, revisar y categorizar los vídeos de manera colaborativa.

La implementación de una plataforma colaborativa que presente vídeos aleatorios para su evaluación, recopile múltiples opiniones y elabore a partir de ellas un consenso, no solo mejoraría la calidad de la categorización, sino que también facilitaría la identificación y exclusión de acciones polémicas o ambiguas.

La obtención de un modelo más potente y preciso, resultado de estas mejoras, tendría implicaciones significativas en el ámbito del arbitraje de esgrima, ya que podría reducir sustancialmente la subjetividad y disparidad en las decisiones arbitrales y constituir una herramienta de apoyo real para los árbitros, aumentando la confianza en las decisiones tomadas durante los combates y contribuyendo a un arbitraje más justo y consistente.

En conclusión, aunque el proyecto actual ha enfrentado limitaciones significativas, las oportunidades de mejora identificadas ofrecen un camino claro hacia el desarrollo de un sistema más robusto y eficaz. La implementación de estas mejoras, incluyendo la ampliación y diversificación del conjunto de datos, la mejora en la detección del florete, y la incorporación de múltiples expertos en el proceso de validación, son pasos cruciales para alcanzar los objetivos de precisión y consistencia en el arbitraje automatizado de esgrima. Estas mejoras no solo tienen el potencial de avanzar significativamente en la automatización del arbitraje en este deporte, sino que también podrían sentar las bases para aplicaciones similares en otros deportes.

Capítulo 7 Bibliografía

- [1] Ramirez Larena, A. (2024). Repositorio Proyecto de Arbitraje de Esgrima con IA. GitHub. Recuperado de <https://github.com/EXCALOFRIO/FENCINGLY.AI>
- [2] Ramirez Larena, A. (2024). Cuaderno Google Colab. Google Colab. Recuperado de <https://colab.research.google.com/drive/1jddqiglpZfFwpNq6eVYp6iPqs3RaW2UL>
- [3] Fencing Database. (n.d.). Recuperado de <https://fencingdatabase.com>
- [4] Wikipedia. (2024). Esgrima. Recuperado de <https://es.wikipedia.org/wiki/Esgrima>
- [5] Ramirez Larena, A. (2024). Script limpiador poses intrusas. Recuperado de <https://github.com/EXCALOFRIO/FENCINGLY.AI/blob/main/src/utils/josnProcesClean.py>
- [6] Amber Roberts arize. (2023). Where To Use Log Loss In Model Monitoring. Recuperado de <https://arize.com/blog-course/binary-cross-entropy-log-loss>
- [7] CMU Perceptual Computing Lab. (n.d.). OpenPose documentation. Recuperado de https://cmu-perceptual-computing-lab.github.io/openpose/web/html/doc/md_doc_00_index.html
- [8] Wu, C.-H., Wu, T.-C., & Lin, W.-B. (2023). Exploration of applying pose estimation techniques in table tennis.
- [9] Ju, C.-Y., Kim, J.-H., & Lee, D.-H. (2023). GolfMate: Enhanced Golf Swing Analysis Tool through Pose Refinement Network and Explainable Golf Swing Embedding for Self-Training.
- [10] Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.
- [11] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735-1780.
- [12] Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. In Advances in Neural Information Processing Systems.

- [13] Bergstra, J., et al. (2013). Hyperopt: A Python library for optimizing the hyperparameters of machine learning algorithms. *Proceedings of the 12th Python in Science Conference*, 13-20.
- [14] Srivastava, N., et al. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*.
- [15] Cao, Z., Simon, T., Wei, S. E., & Sheikh, Y. (2017). Realtime multi-person 2D pose estimation using part affinity fields. In *CVPR 2017*.
- [16] Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., ... & Kudlur, M. (2016). TensorFlow: A system for large-scale machine learning. In *OSDI'16: Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation*, 265-283.
- [17] Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2623-2631.
- [18] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- [19] Smith, J. (2020). Binary Cross-Entropy Loss for Classification Problems. *Journal of Machine Learning Research*, 21(3), 123-145.
- [20] Federación Internacional de Esgrima. (2020). Reglamento

Glosario de términos

Inteligencia Artificial (IA): Rama de la informática que se ocupa de la creación de sistemas capaces de realizar tareas que normalmente requieren inteligencia humana, como el aprendizaje, la resolución de problemas y la toma de decisiones.

Vídeo Arbitraje: Sistema que utiliza grabaciones de vídeo para ayudar a los árbitros en la toma de decisiones durante competiciones deportivas, especialmente para revisar jugadas dudosas o determinar si una acción cumple con las reglas.

Esgrima: Deporte de combate en el que dos contrincantes se enfrentan con armas blancas, como el florete, la espada o el sable, con el objetivo de tocar al oponente.

Florete: Arma blanca de hoja larga y flexible, con la punta protegida por una almohadilla, utilizada en esgrima. En el florete, los tocados solo son válidos en el torso.

Aprendizaje Profundo (Deep Learning): Tipo de aprendizaje automático que utiliza redes neuronales con múltiples capas para analizar datos y realizar tareas complejas.

Precisión: Capacidad de un modelo de aprendizaje automático para realizar predicciones correctas. Se mide como la proporción de predicciones correctas sobre el total de predicciones realizadas.

Consistencia: Capacidad de un modelo de aprendizaje automático para producir resultados similares o coherentes en diferentes momentos y con diferentes conjuntos de datos.

Decisiones Arbitrales: Decisiones tomadas por un árbitro durante una competición deportiva para hacer cumplir las reglas del juego y determinar el resultado de las acciones.

OpenPose: Herramienta de código abierto para la detección de poses humanas en imágenes y vídeos. Utiliza técnicas de aprendizaje profundo para identificar y seguir los puntos clave del cuerpo humano.

Redes Neuronales: Modelos computacionales inspirados en el cerebro humano, formados por nodos interconectados que procesan y transmiten información.

LSTM (Long Short-Term Memory): Tipo de red neuronal recurrente diseñada para aprender dependencias a largo plazo en secuencias de datos, como vídeos o texto.

Capas Densas: Capas de una red neuronal en la que cada nodo está conectado a todos los nodos de la capa anterior.

Data Augmentation: Técnica utilizada para aumentar la cantidad y diversidad de datos de entrenamiento mediante la generación de variantes artificiales de los datos originales.

Overfitting: Fenómeno que ocurre cuando un modelo de aprendizaje automático se ajusta demasiado bien a los datos de entrenamiento, memorizándolos en lugar de aprender patrones generales, lo que resulta en un bajo rendimiento en datos no vistos.

TensorFlow: Biblioteca de código abierto para aprendizaje automático desarrollada por Google, utilizada para construir y entrenar modelos de aprendizaje profundo.

Optuna: Biblioteca de código abierto para la optimización de hiperparámetros en modelos de aprendizaje automático.

Binary Cross-Entropy: Función de pérdida utilizada en problemas de clasificación binaria para medir la diferencia entre las predicciones del modelo y las etiquetas reales.

AUC (Area Under the ROC Curve): Métrica utilizada para evaluar el rendimiento de modelos de clasificación binaria. Mide la capacidad del modelo para distinguir entre las dos clases.

F1 Score: Métrica que combina la precisión y la exhaustividad (recall) en una sola medida. Se utiliza para evaluar el rendimiento general de un modelo de clasificación.